Marketplace Payment Spec

10k Athletics — Creator Marketplace v1

Owner: Blake Wellington · Status: Implemented (v1) · Last updated: April 2026

Context

10k Athletics runs a creator marketplace where coaches, trainers, and content creators sell digital programs and services to athletes. Players purchase inside the player app or web platform; creators receive payouts via Stripe Connect after platform commission.

Stripe Connect handles the rails: card processing, KYC, payout mechanics, dispute primitives. It does not, on its own, answer the questions a marketplace has to resolve at the application layer:

- Who is allowed to list and receive payouts, and at what verification state.
- When a payout actually clears versus when it is held.
- How the platform recovers money it has fronted on a creator's behalf (the $500 setup-service balance).
- How disputes, refunds, and mid-purchase tier changes are handled without breaking trust on either side.

This document is the operating spec engineering builds against. It defines the money flow, the data model, the business rules for edge cases, and the explicit build-vs-buy boundary between Stripe and 10k.

Goals & Non-Goals

Goals (v1)

- Onboard creators with KYC complete before any payout is possible.
- Process player purchases end-to-end with platform commission split correctly at the time of sale.
- Recover the $500 platform setup-service balance from a creator's first marketplace earnings before any payout.
- Handle the four most common edge cases in v1 (incomplete onboarding, refunds, mid-purchase tier upgrades, suspended creators) without manual intervention.
- Keep the rules legible: a non-engineer should be able to read this doc and predict what the system will do.

Non-Goals (v1)

- Multi-currency payouts. USD only at launch; Sheldon's audience is primarily US-based.
- Tax form generation beyond Stripe's defaults (1099-K via Stripe Connect Express is sufficient).
- Subscription billing for creator products. v1 ships one-time purchases only; subscriptions deferred to v1.1.
- Chargeback automation beyond Stripe's defaults. Disputes route to a manual queue in v1.

Money Flow

Every dollar that enters the system is in one of six states. The goal of this section is to make those states explicit so the data model and business rules can reference them by name.

State	Definition	Who holds it / what unblocks the next state
AUTHORIZED	Player's card has been authorized but not captured.	Stripe holds. Transitions to CAPTURED on successful charge or VOID on failure.
CAPTURED	Funds settled to platform Stripe balance. Commission split recorded in our DB.	Platform Stripe account holds. Transitions to PENDING_PAYOUT after the dispute window (T+2 days) if no chargeback.
PENDING_PAYOUT	Cleared dispute window. Eligible for payout to creator's connected account.	Platform holds. Transitions to PAID_OUT on weekly payout run, or to OFFSET if creator has outstanding setup balance.
OFFSET	Earnings applied against the creator's $500 setup-service balance.	Platform retains. Setup balance ledger debited. Once balance reaches zero, residual rolls to PENDING_PAYOUT.
PAID_OUT	Funds transferred to creator's connected Stripe account.	Creator holds (subject to Stripe's own payout schedule to bank).
CLAWED_BACK	Dispute or refund issued after payout. Platform owes Stripe; creator owes platform.	Recovery rules in §Refunds & Disputes. Creator account flagged if balance is negative.

The state names above are used as-is in the database (transactions.state) so engineering, support, and finance can reason about the same thing with the same vocabulary.

Data Model

Two primary entities. Both intentionally lean — most identity, payout, and dispute data lives in Stripe and is referenced by ID rather than mirrored locally.

creators

Field	Type	Notes
id	uuid	Primary key. Internal reference only.
stripe_account_id	string	Stripe Connect account ID. Source of truth for KYC.
verification_status	enum	PENDING | VERIFIED | RESTRICTED | SUSPENDED. Mirrored from Stripe webhook for read performance.
payout_eligible	bool	Computed: verification_status == VERIFIED AND charges_enabled == true AND payouts_enabled == true.
setup_balance_cents	int	Outstanding $500 setup-service debt. Decrements via OFFSET state. Cannot go negative.

Field	Type	Notes
commission_rate_bps	int	Basis points. Default 2000 (20%). Per-creator overrides for strategic partners (Sheldon: 1500).
listing_enabled	bool	Independent of payout_eligible. Creator can list while KYC is pending; cannot get paid until verified.

transactions

Field	Type	Notes
id	uuid	Primary key.
stripe_payment_intent_id	string	Source of truth for payment status. Idempotency key on retries.
creator_id	uuid	FK to creators.
player_id	uuid	FK to users.
gross_cents	int	Amount the player paid before any split.
commission_cents	int	Platform's cut. Computed at time of sale using creator's then-current commission_rate_bps. Locked once written.
creator_cents	int	gross_cents - commission_cents - stripe_fee_cents.
state	enum	AUTHORIZED | CAPTURED | PENDING_PAYOUT | OFFSET | PAID_OUT | CLAWED_BACK. See §Money Flow.
payout_id	string?	Stripe payout ID once funds reach creator. Null until PAID_OUT.

Two design decisions worth flagging:

- **commission_cents is locked at sale time.** If a creator's rate changes (strategic partner promotion, for example), in-flight transactions keep the old rate. This avoids retroactive rewrites that confuse both creators and finance.
- **verification_status is mirrored from Stripe, not owned by us.** Stripe is source of truth via webhook; the local field exists only for read performance on listing pages. If the webhook is ever in question, refetch from Stripe before making a payout decision.

Business Rules

These are the rules the application enforces on top of Stripe. Each rule names the failure mode it prevents.

Listing & verification

- **R1.** A creator may list products as soon as their account is created (listing_enabled = true). They may not receive payouts until verification_status == VERIFIED.
- **R2.** Sales made while verification_status != VERIFIED are accepted and held in CAPTURED state. They transition to PENDING_PAYOUT only after the creator completes KYC. (Rationale: do not punish creators or buyers for slow KYC. Funds are safe in the platform balance until then.)
- **R3.** If verification_status flips to RESTRICTED or SUSPENDED, listing_enabled is automatically set to false. Existing in-flight transactions resolve under their original rules; no new sales accepted.

Setup balance recovery

- **R4.** While setup_balance_cents > 0, all earnings route to OFFSET instead of PENDING_PAYOUT. Earnings reduce the balance; once balance == 0, residual on the same transaction rolls forward to PENDING_PAYOUT.
- **R5.** A creator can see their setup balance and progress against it in their dashboard at all times. (Surfacing this prevents the most predictable support ticket: "Why haven't I been paid?")

Refunds & disputes

- **R6.** Refunds before PAID_OUT: reverse the transaction, return funds to player, decrement creator earnings owed. No clawback needed because no payout occurred.
- **R7.** Refunds after PAID_OUT: transaction transitions to CLAWED_BACK. Platform refunds the player from its own balance and creates a negative balance on the creator. The next earnings offset that balance before any new payout. (Rationale: do not pull from the creator's bank account; recover from forward earnings.)
- **R8.** Stripe disputes route to a manual queue. v1 does not auto-respond to disputes; a human reviews evidence within 48 hours. (Trade-off: low volume in v1 makes manual review cheap and lets us learn the dispute patterns before automating.)

Mid-purchase tier changes

- **R9.** If a creator changes a product's price between the time a player adds it to cart and the time they check out, the price at checkout is the price the player is charged. The cart is re-priced at the checkout step and shown to the player before payment.
- **R10.** Tier upgrades within an active product (e.g., basic → premium) are treated as new transactions, not modifications. This avoids partial-refund accounting and keeps the transaction ledger immutable.

Build vs. Buy: What Stripe Owns, What 10k Owns

Explicit boundary. The point of this table is to make the trade-off legible — every row is a place where we could have built more and chose not to, or could have leaned harder on Stripe and chose not to.

Concern	Owner	Why
Card processing, PCI scope	Stripe	Zero reason to touch card data ourselves. Stripe Elements keeps us out of PCI scope entirely.
KYC / identity verification	Stripe (Connect Express)	Regulated, painful, and not differentiating. Stripe's flow is

Concern	Owner	Why
		good enough and gets us 1099-K generation for free.
Payout mechanics to bank	Stripe	ACH, holds, micro-deposits — Stripe's problem. We trigger payouts; they execute.
Dispute primitives (chargeback, refund APIs)	Stripe	We use the APIs; we don't reimplement them.
Listing eligibility	10k	Stripe's verification state isn't sufficient — we have product-level rules (listing while pending, suspending bad actors).
Commission split logic	10k	Per-creator rate overrides + locked-at-sale-time semantics are 10k policy, not Stripe's.
Setup-balance offset against earnings	10k	Specific to our business model. Stripe doesn't know what a setup-service debt is.
Dispute review (v1)	10k (manual)	Volume is low. Build automation when patterns emerge, not before.
Clawback after refund-post-payout	10k	Stripe can pull from a connected account, but we'd rather offset forward earnings than touch a creator's bank.

The pattern: Stripe owns anything regulated, infrastructural, or commodity. 10k owns anything that encodes our specific policy or trust relationship with creators.

Open Questions

Items deliberately left unresolved in v1, with the trigger that should force a decision:

- **Subscription billing.** Deferred to v1.1. Trigger: first creator who wants to sell a recurring program rather than a one-time pack.
- **Multi-currency payouts.** Deferred. Trigger: signed creator with non-US audience >25% of revenue.
- **Dispute automation.** Deferred. Trigger: dispute volume >5/month, or pattern recognition becomes worth encoding.
- **Negative-balance creators.** Current rule (R7) recovers from forward earnings. Open question: what happens if a creator with a negative balance never sells again? v1 accepts the loss. Worth revisiting if it becomes more than rounding.

Notes for Engineering

- All money fields are integer cents. No floats anywhere in the payment path.
- Idempotency: stripe_payment_intent_id is the idempotency key for retries. Never create a second PaymentIntent for the same cart.
- Webhooks are the source of truth for state transitions. Polling is allowed for reconciliation but never as primary signal.
- All state transitions are logged to a transaction_events table (append-only) for audit and support.